



Warsaw University of Technology

**Faculty of Mathematics
and Information Science**

Master's diploma thesis

in the field of study

Data Science

Research on Model Compression and Efficiency Optimization
Techniques on Deep Learning Models for Edge Deployment

Hubert Jaczyński

student record book number 320597

thesis supervisor

dr hab. Maria Ganzha, prof. PW

WARSAW 2026

Abstract

Research on Model Compression and Efficiency Optimization Techniques on Deep Learning Models for Edge Deployment

Deep learning models achieve high accuracy in computer vision, but their computational and memory demands often prevent direct deployment on resource-constrained edge devices such as cameras, robots, drones, and mobile platforms. This thesis examines how to make modern object detection models suitable for real-time on-device inference by reducing their computational cost while maintaining detection quality. The work reviews the main architectural families used in vision (convolutional networks and transformer-based models) and then focuses on practical efficiency methods applicable to edge deployment. In particular, it studies model compression and acceleration techniques, including quantisation (post-training and quantisation-aware training), pruning (structured and unstructured), and knowledge distillation, as well as architecture-level approaches such as hardware-aware neural architecture search. The proposed evaluation emphasises realistic deployment conditions and compares methods using consistent metrics, including detection accuracy, inference latency, and model size. Where feasible, it also considers energy-related aspects. The overall goal is to identify which techniques, or combinations of techniques, provide the most favourable accuracy-latency-size trade-off for real-time object detection on constrained hardware and how strongly the results depend on the target device and inference runtime.

Keywords: Model compression, efficiency optimisation, deep learning, neural networks, quantisation, pruning, knowledge distillation, hardware-aware neural architecture search, edge device deployment

Streszczenie

Badania nad technikami kompresji modeli i optymalizacji wydajności modeli głębokiego uczenia się do wdrażania na urządzeniach brzegowych

Modele głębokiego uczenia się osiągają wysoką dokładność w zakresie widzenia komputerowego, ale ich wymagania obliczeniowe i pamięciowe często uniemożliwiają bezpośrednie wdrożenie na urządzeniach brzegowych o ograniczonych zasobach, takich jak kamery, roboty, drony i platformy mobilne. Niniejsza praca bada, w jaki sposób dostosować nowoczesne modele wykrywania obiektów do wnioskowania w czasie rzeczywistym na urządzeniach, zmniejszając ich koszty obliczeniowe przy zachowaniu jakości wykrywania. W pracy dokonano przeglądu głównych rodzin architektur stosowanych w wizji (sieci konwulcyjne i modele oparte na transformatorach), a następnie skupiono się na praktycznych metodach zwiększania wydajności, które można zastosować w przypadku wdrażania na urządzeniach brzegowych. W szczególności zbadano techniki kompresji i przyspieszania modeli, w tym kwantyzację (po szkoleniu i szkolenie uwzględniające kwantyzację), przycinanie (ustrukturyzowane i nieustrukturyzowane) oraz destylację wiedzy, a także podejścia na poziomie architektury, takie jak wyszukiwanie architektury neuronowej uwzględniające sprzęt. Proponowana ocena kładzie nacisk na realistyczne warunki wdrożenia i porównuje metody przy użyciu spójnych wskaźników, w tym dokładności wykrywania, opóźnienia wnioskowania i rozmiaru modelu. Tam, gdzie to możliwe, uwzględnia również aspekty związane z energią. Ogólnym celem jest określenie, które techniki lub kombinacje technik zapewniają najkorzystniejszy kompromis między dokładnością, opóźnieniem i rozmiarem w przypadku wykrywania obiektów w czasie rzeczywistym na ograniczonym sprzęcie oraz w jakim stopniu wyniki zależą od urządzenia docelowego i czasu działania wnioskowania.

Słowa kluczowe: Kompresja modeli, optymalizacja wydajności, głębokie uczenie się, sieci neuronowe, kwantyzacja, przycinanie, destylacja wiedzy, wyszukiwanie architektur neuronowych uwzględniających sprzęt, wdrażanie na urządzeniach brzegowych

Contents

1. Introduction	11
1.1. Aim of the work	12
2. Related work	14
2.1. Neural networks	14
2.1.1. Training neural networks	15
2.2. Deep neural networks	17
2.3. Convolutional neural networks	18
2.3.1. Convolution operation	18
2.3.2. Pooling and hierarchical representations	19
2.3.3. Typical CNN building blocks	19
2.3.4. Efficiency considerations for edge deployment	19
2.3.5. CNNs and model compression	20
2.4. Transformers	20
2.4.1. Self-attention mechanism	20
2.4.2. Transformer block	22
2.4.3. Efficiency considerations	23
2.4.4. Relation to computer vision	23
2.4.5. Vision Transformers	23
2.5. Frugal artificial intelligence	24
2.5.1. Quantisation	24
2.5.2. Pruning	27
2.5.3. Knowledge distillation	30
2.5.4. Neural Architecture Search and Hardware-aware design	32

1. Introduction

Deep learning is used in many everyday technologies, especially in systems that analyse images and video [13]. The models behind these systems are improving quickly, but they are also becoming larger and more demanding to run. While powerful servers can handle such models, many real-world applications require the model to run directly on a small device, such as in cameras, robots, drones, or mobile systems [11]. These devices have limited computing power, memory, and battery capacity, so a model that performs well on a workstation may be too slow, too large, or too energy-intensive to be practical on the device [25].

For this reason, there is a growing need to make models lighter and faster while keeping their results as reliable as possible [4]. In practice, this means balancing several factors: the model’s accuracy, its prediction speed, the memory it requires, and the energy it consumes. The best balance depends on the intended use case — for example, a safety-critical system may prioritise accuracy, while a battery-powered device may prioritise speed and energy efficiency.

To address these challenges, researchers develop methods that reduce the computational and memory requirements of a model, while aiming to preserve performance as much as possible. Some approaches simplify the model structure, while others reduce the precision of computations, and still others transfer knowledge from a larger model to a smaller one [8]. Additionally, practical performance also depends on the specific hardware and software used to run the model, so it is essential to evaluate solutions under realistic deployment conditions [3].

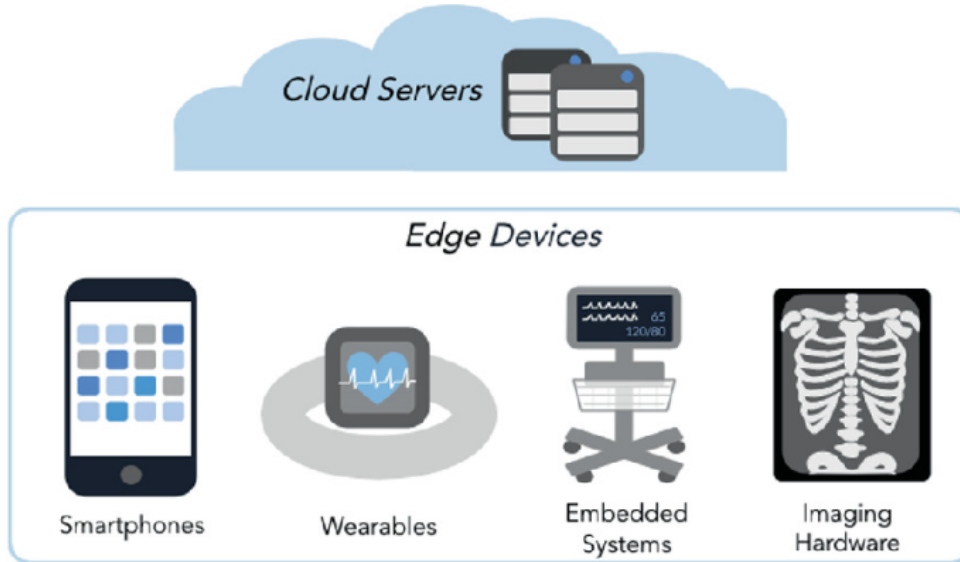


Figure 1.1: Variants of edge devices (reproduced from [1]).

The use of these approaches may have an impact on the application of artificial intelligence in various scenarios. Some of them may not yield any benefit where the highest accuracy is required. Compression may be unacceptable without strict validation in safety-critical settings (eg. medical, automotive).

1.1. Aim of the work

This thesis focuses on real-time object detection on resource-limited devices. The goal is to investigate and compare methods that reduce the computational and memory requirements of object detection models, enabling them to run efficiently on edge hardware without compromising detection quality. The work will review modern approaches used for on-device deployment, implement selected solutions, and evaluate them using consistent and practical metrics such as detection accuracy, inference latency, model size, and (where possible) energy-related measurements.

The study is guided by the following research questions:

1. Which model architectures are the most appropriate for deployment on resource-limited devices when considering the trade-off between accuracy and computational cost?
2. Which methods for making models smaller and faster, or combinations of such methods, provide the best trade-off between accuracy, inference speed, model size, and energy usage on edge devices?

1.1. AIM OF THE WORK

3. How do post-training quantisation and quantisation-aware training differ in their impact on object detection accuracy and inference latency?
4. Under what conditions can training a smaller model to mimic a larger one provide better results than relying only on direct training or model simplification?
5. To what extent do the final results depend on the target hardware and the software stack used for deployment?

2. Related work

The topic addressed in this work belongs to one of the most actively researched areas in modern machine learning. As deep learning systems are deployed more widely, researchers increasingly study how far such models can be made smaller, faster, and less resource-demanding while still producing results that remain useful in practice [4]. This need is driven not only by scientific interest, but also by real engineering constraints: reduced computation can lower deployment costs, enable real-time operation, and make advanced models accessible on devices with limited power, memory, and processing capability [25]. At the same time, the rapid pace of technological change means that the limits of what is feasible continue to shift as new model architectures, optimisation techniques, and hardware platforms emerge [26].

In this thesis, artificial intelligence is considered primarily as a practical tool for automating perception tasks that would otherwise require significant human effort. Specifically, computer vision methods can be employed to detect, classify, or segment objects in images and videos [21, 16]. While these tasks may appear straightforward for humans in many situations, robust performance across different environments, lighting conditions, viewpoints, and object scales remains challenging for machines. Modern deep learning approaches have achieved strong results, but they often require substantial computational resources, which motivates research on methods that reduce inference cost without sacrificing too much accuracy.

The purpose of the related works chapter is therefore to summarise the key directions of research that address efficient inference and deployment. It discusses commonly used approaches to improve efficiency, how these methods affect accuracy and runtime behaviour, and what has been reported in the literature when such techniques are applied to real-time object detection on constrained hardware [20, 8].

2.1. Neural networks

Neural networks are inspired by the way biological nervous systems process information. In their simplest form, they consist of many small computational units (often called *neurons*) that

transform inputs into outputs and can be connected into larger structures. One of the earliest and most influential mathematical models of an artificial neuron was proposed by McCulloch and Pitts in 1943 [17]. In this model, a neuron acts as a threshold-based logic unit: it sums its inputs and produces an output depending on whether that sum exceeds a chosen threshold [17]. This idea later became a foundation for trainable models such as the perceptron and, over time, for more complex architectures [23].

As computing power and training methods improved, neural networks evolved into many specialised families. Examples include recurrent neural networks (RNNs) for sequential data [10], convolutional neural networks (CNNs) for images [14], generative adversarial networks (GANs) for data generation [7], and transformer-based models, which currently dominate many state-of-the-art results in both vision and language tasks [27].

2.1.1. Training neural networks

Training a neural network means finding parameter values (weights and biases) that minimise a chosen objective function [13]. In supervised learning, this objective typically measures the discrepancy between the model predictions and the ground-truth labels. Let a model with parameters θ produce predictions $f(\mathbf{x}; \theta)$ for an input $\mathbf{x}$. Given a dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ and a loss function $\ell(\cdot)$, the learning problem is commonly written as

$$\min_{\theta} \mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f(\mathbf{x}_i; \theta), y_i),$$

possibly with an additional regularisation term, e.g., $\lambda \|\theta\|_2^2$, to discourage overly large weights and improve generalisation.

Gradient-based optimisation

Modern neural networks are trained using gradient-based methods [13]. Gradients $\nabla_{\theta} \mathcal{L}(\theta)$ are computed efficiently with backpropagation, which applies the chain rule through the computational graph of the network [24]. A basic optimisation method is gradient descent, which iteratively updates parameters as

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t),$$

where η is the learning rate. In practice, evaluating the full gradient over all N samples at every step is expensive, so training is usually performed with *stochastic gradient descent* (SGD) or its variants [22].

Stochastic gradient descent

In SGD, the gradient is estimated using a mini-batch $\mathcal{B}_t \subset \mathcal{D}$:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}_{\mathcal{B}_t}(\theta_t), \quad \mathcal{L}_{\mathcal{B}_t}(\theta) = \frac{1}{|\mathcal{B}_t|} \sum_{(\mathbf{x}, y) \in \mathcal{B}_t} \ell(f(\mathbf{x}; \theta), y).$$

The stochasticity introduced by mini-batches makes optimisation noisier, but also helps escape poor local regions and enables training on large datasets. Common improvements to SGD include momentum and weight decay. Momentum accumulates a running direction of descent [19]:

$$\mathbf{v}_{t+1} = \mu \mathbf{v}_t + \nabla_{\theta} \mathcal{L}_{\mathcal{B}_t}(\theta_t), \quad \theta_{t+1} = \theta_t - \eta \mathbf{v}_{t+1},$$

where $\mu \in [0, 1)$ controls how strongly past gradients influence the update.

The convergence to the minimum is shown in Figure 2.1

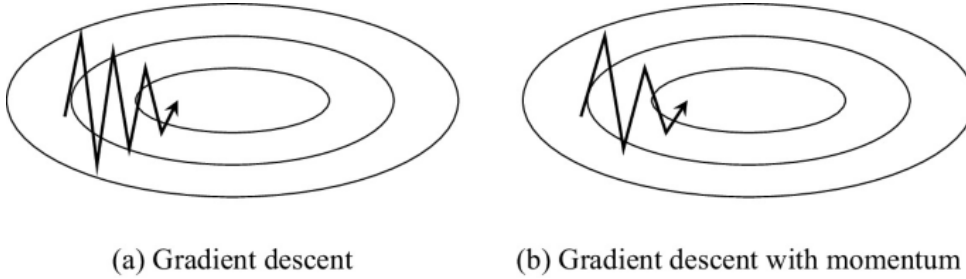


Figure 2.1: Stochastic Gradient Descent comparison (reproduced from [18]).

Learning rate selection and line search intuition

The learning rate is one of the most important hyperparameters: if it is too large, the optimisation may diverge; if it is too small, convergence can be slow [13]. In classical optimisation, *line search* methods choose a step size that sufficiently decreases the objective (for example, through the Armijo condition). Although full line search is rarely used in large-scale deep learning due to its computational cost, the same intuition is evident in practice through learning rate schedules, warm-up phases, and adaptive strategies.

A related idea is *backtracking*: when an update increases the loss or causes unstable training, the step size is reduced and the update is retried. While this is not always implemented explicitly, it is conceptually similar to common training practices such as reducing the learning rate on a plateau, using gradient clipping to avoid exploding updates, or using early stopping to prevent overfitting [13].

Searching for good solutions

Neural network training is a non-convex optimisation problem, meaning there may be many parameter configurations that achieve low loss but differ in generalisation performance and robustness. As a result, training often involves a search process over:

- **Optimiser choice** (e.g., SGD with momentum, Adam, AdamW),
- **Learning rate schedules** (step decay, cosine decay, one-cycle policy),
- **Regularisation** (weight decay, data augmentation, dropout),
- **Model capacity** (number of layers/channels, feature dimensions).

These choices influence not only final accuracy but also how well a model transfers to constrained environments, such as edge devices [25].

2.2. Deep neural networks

A deep neural network is a neural network composed of multiple successive layers, where each layer applies a learned transformation followed by a non-linear activation [13]. In contrast to shallow models, depth allows the network to represent complex functions through the composition of simpler operations. In computer vision, this often leads to hierarchical feature representations: early layers learn low-level patterns, such as edges and textures, while deeper layers capture higher-level structures, including object parts and entire objects [13].

Despite their strong performance, deep models can be computationally demanding [25]. The cost of inference is determined not only by the number of parameters (which affects storage) but also by the number of operations and the memory required to store intermediate activations. In edge deployments, limitations in processing power, memory capacity, and energy availability make these factors critical. In particular, memory access and bandwidth can become a bottleneck, meaning that reductions in model size or operation count do not always translate directly into proportional speed-ups on a given device [25].

These properties motivate the use of efficiency techniques that aim to reduce resource requirements while preserving accuracy [4]. Methods such as reduced numerical precision, removal of redundant parameters or structures, transferring knowledge from larger models, and automatically searching for efficient architectures are therefore central to deploying deep networks under real-world constraints [6, 8].

2.3. Convolutional neural networks

Convolutional Neural Networks (CNNs) are a class of deep learning models designed primarily for processing data with a grid-like structure, most notably images [13]. Their key idea is to exploit the spatial structure of an image by using *local connectivity* and *weight sharing*. Unlike fully connected layers, which learn a separate weight for every input feature, convolutional layers apply the same small set of weights (a kernel) across the entire image. This reduces the number of parameters and makes the model more efficient, while also providing a degree of *translation equivariance*, meaning that shifting an object in the image results in a corresponding shift in the feature map. An example architecture is shown in Figure 2.2.

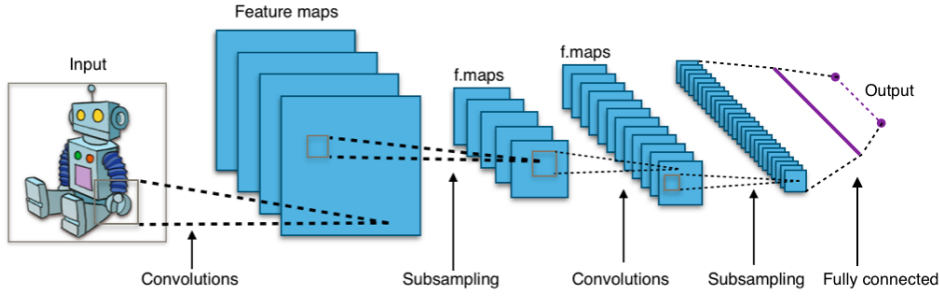


Figure 2.2: CNN architecture visualisation. Reproduced from Aphex34 [2] (CC BY-SA 4.0).

2.3.1. Convolution operation

Given an input feature map $X \in \mathbb{R}^{H \times W \times C_{\text{in}}}$, a convolutional layer produces an output feature map $Y \in \mathbb{R}^{H' \times W' \times C_{\text{out}}}$ by sliding learnable filters over the spatial dimensions. For a filter of size $k \times k$, the output at spatial location (u, v) and output channel j can be written as

$$Y_{u,v,j} = b_j + \sum_{c=1}^{C_{\text{in}}} \sum_{p=0}^{k-1} \sum_{q=0}^{k-1} W_{p,q,c,j} X_{u+p,v+q,c}, \quad (2.1)$$

where W denotes the convolutional kernel weights and b_j is the bias for channel j . In practice, convolutions often use a stride s (skipping positions) and padding P (extending borders) to control the spatial resolution of the output [13].

Convolution is usually followed by a non-linear activation such as ReLU,

$$\text{ReLU}(x) = \max(0, x),$$

which enables the network to model non-linear decision functions. Stacking multiple convolutional layers allows the network to build increasingly abstract features.

2.3.2. Pooling and hierarchical representations

Many CNN architectures include pooling operations, such as max pooling, to reduce spatial resolution and increase robustness to small shifts and distortions [13]. For example, max pooling with window size $k \times k$ can be expressed as

$$Y_{u,v,c} = \max_{(p,q) \in \Omega} X_{u+p,v+q,c},$$

where Ω indexes the pooling window. Modern architectures may replace pooling with strided convolutions, but the purpose is similar: gradually reducing resolution while increasing the number of channels, forming a hierarchical representation [13].

2.3.3. Typical CNN building blocks

Over time, CNNs have evolved from simple stacks of convolution and pooling layers into more advanced building blocks [13]:

- **Residual connections** (e.g., ResNet), which add skip paths to improve gradient flow and enable deeper networks.
- **Normalisation layers** (e.g., Batch Normalisation), which stabilise training and often speed up convergence.
- **Bottleneck designs**, which use 1×1 convolutions to reduce and then restore channel dimensionality, decreasing computation.

These design choices are important for practical efficiency, especially when models are deployed on limited hardware [25].

2.3.4. Efficiency considerations for edge deployment

CNNs are widely used for edge inference because their operations are structured and can be implemented efficiently on GPUs, NPUs, and other accelerators [25]. However, CNNs can still be computationally expensive, particularly in early layers that process high-resolution feature maps. The total inference cost is influenced by:

- the input resolution (H, W) and the number of channels,
- kernel size and stride,
- the number of layers and output channels,
- memory bandwidth required to read and write feature maps.

A common proxy for compute is the number of multiply-accumulate operations (MACs) [25]. For a standard 2D convolution, the approximate MAC count is

$$\text{MACs} \approx H'W' \cdot C_{\text{out}} \cdot (k^2 C_{\text{in}}).$$

This expression highlights why reducing resolution, kernel size, or channel counts can strongly affect runtime [25].

2.3.5. CNNs and model compression

CNN structure makes them suitable for many compression methods discussed later in this thesis [4]. For example, *structured pruning* can remove entire channels or filters, directly reducing computation [8]. *Quantisation* can reduce the precision of weights and activations (e.g., from FP32 to INT8), lowering memory usage and often improving latency on supported hardware [6]. Furthermore, efficient CNN variants (e.g., based on depthwise separable convolutions and other factorised operations) are often designed specifically to reduce MACs and memory access, which is particularly beneficial for real-time object detection on edge devices [11].

Overall, CNNs remain a foundational architecture in computer vision. Even when newer paradigms, such as transformers, are used, CNN-like components are often retained in early stages or in hybrid models due to their efficiency and strong inductive bias for local spatial patterns [13].

2.4. Transformers

Transformers are a class of neural network architectures originally introduced for sequence modelling tasks [27]. Their main idea is to replace recurrence with an attention mechanism that enables the model to directly relate different parts of the input to one another. This makes transformers highly parallelisable during training and enables them to capture long-range dependencies more effectively than many earlier architectures [27].

2.4.1. Self-attention mechanism

The core component of a transformer is *self-attention* [27]. Given an input sequence represented by embeddings

$$X \in \mathbb{R}^{n \times d},$$

2.4. TRANSFORMERS

where n is the number of tokens and d is the embedding dimension, the model computes three learned projections: queries Q , keys K , and values V :

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V,$$

where $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k}$ are trainable matrices. Attention weights are then computed using scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V. \quad (2.2)$$

Intuitively, the softmax term determines how strongly each token should attend to all other tokens. This allows the representation of each token to be updated based on information from the entire sequence [27].

In practice, transformers use *multi-head attention*, where multiple attention operations are performed in parallel with different learned projections [27]. This enhances expressiveness by enabling the model to simultaneously focus on different types of relationships. The outputs of the heads are concatenated and projected back to the model dimension:

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O.$$

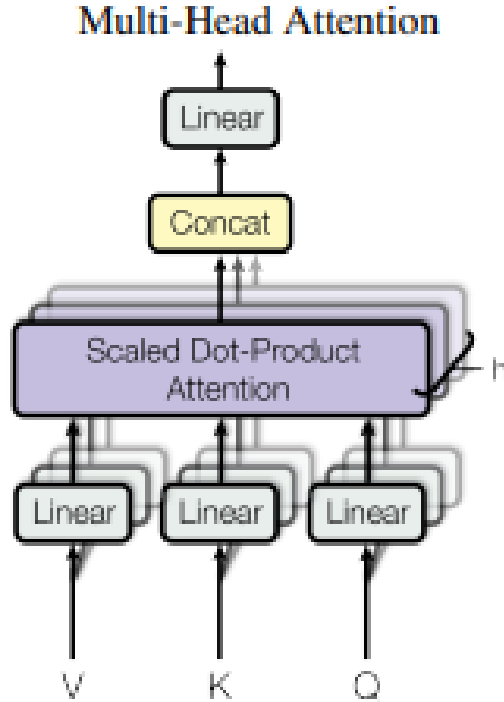


Figure 2.3: Multi-head attention overview (reproduced from [27]).

2.4.2. Transformer block

A standard transformer encoder block consists of:

- a multi-head self-attention layer,
- a position-wise feed-forward network (FFN),
- residual connections and normalisation layers.

A simplified formulation of an encoder block can be written as

$$X' = \text{LN}(X + \text{MHA}(X)), \quad Y = \text{LN}(X' + \text{FFN}(X')),$$

where $\text{LN}(\cdot)$ denotes layer normalization. The feed-forward network is typically implemented as two linear layers with a non-linear activation:

$$\text{FFN}(x) = W_2 \sigma(W_1 x + b_1) + b_2.$$

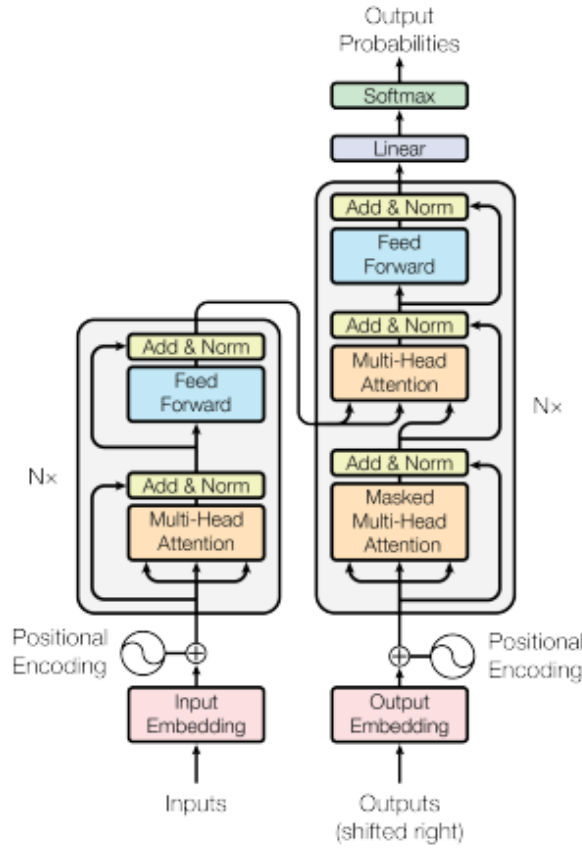


Figure 2.4: Transformer architecture overview (reproduced from [27]).

2.4.3. Efficiency considerations

Compared to convolutions, the attention mechanism has a different computational profile [27]. For a sequence of length n , the QK^T operation in Equation (2.2) has $\mathcal{O}(n^2)$ complexity, which can become expensive when n is large [27]. In many practical settings, runtime and memory are influenced not only by arithmetic operations but also by the cost of moving data between memory and compute units [25]. These factors are particularly important for edge deployment, where memory bandwidth and cache sizes are limited [25].

2.4.4. Relation to computer vision

Transformers were originally designed for language processing, where inputs are naturally represented as token sequences [27]. Images, however, are two-dimensional arrays of pixels. To apply transformers to vision tasks, images must be converted into a sequence representation while preserving spatial information. This motivates Vision Transformers, introduced in the next subsection [5].

2.4.5. Vision Transformers

Vision Transformers (ViTs) adapt transformer encoders to image understanding by representing an image as a sequence of fixed-size patches [5]. An image of size $H \times W$ is split into patches of size $P \times P$, producing

$$n = \frac{HW}{P^2}$$

patches. Each patch is flattened and mapped into a d -dimensional embedding space. Because transformers have no inherent notion of token order, learnable *positional embeddings* are added to encode patch locations [5]. The resulting sequence is then processed by a standard transformer encoder stack [5].

ViTs have demonstrated strong performance, particularly when trained on large datasets, and are utilised in many modern vision systems [5]. However, the quadratic cost of attention with respect to the number of patches means that high-resolution inputs can be computationally expensive [27]. As a result, many practical vision transformer variants introduce efficiency improvements, such as hierarchical token representations, local or windowed attention, and reduced token counts [15]. These considerations are directly relevant to edge deployment, where input resolution and model size strongly affect latency and energy consumption [25].

2.5. Frugal artificial intelligence

As deep learning systems become more widely deployed, practical limitations such as computation time, memory usage, energy consumption, and data availability increasingly determine whether a solution can be used in real-world scenarios. In many applications, especially on edge devices, it is not sufficient for a model to be accurate in a laboratory setting—it must also be efficient enough to run under strict hardware constraints and, in some cases, with limited access to training data or communication bandwidth. These constraints motivate a set of methods whose goal is to reduce the overall “cost” of using artificial intelligence while keeping performance as close as possible to that of unconstrained models [4].

In this thesis, the term *frugal artificial intelligence* refers to the design and deployment of AI systems that aim to achieve strong predictive performance while minimising resource requirements [25]. This includes reducing the computational cost of inference, lowering memory and storage demands, and improving energy efficiency [25]. In addition, frugal approaches can also address data-related costs, such as the amount of labelled data needed for training or the cost of transmitting data to remote servers. The underlying idea is to build systems that are not only accurate but also feasible to deploy and maintain in environments with limited resources.

Frugal AI is particularly relevant for real-time computer vision tasks such as object detection, where models must process high-dimensional inputs (images or video frames) within strict time budgets [20]. In such settings, the core challenge is to navigate trade-offs between accuracy and efficiency. The following subsections introduce the main families of techniques used to make deep learning models more suitable for constrained deployment [4]. These include reducing numerical precision, removing redundant parameters or structures, transferring knowledge from larger models to smaller ones, and automatically searching for architectures that satisfy hardware-dependent requirements [6, 9, 3].

2.5.1. Quantisation

Quantisation is a family of techniques that reduce the numerical precision used to represent a neural network. In standard deep learning workflows, model weights and activations are typically stored and computed in 32-bit floating point (FP32). Although FP32 offers high numerical accuracy, it requires significant memory bandwidth and compute [25]. Quantisation replaces FP32 with lower-precision formats such as FP16 or INT8, which can reduce model size and, on supported hardware, improve inference speed and energy efficiency. This makes quantisation particularly relevant for edge deployment, where memory and power are often limited.

Basic idea

In many practical schemes, a real-valued tensor x is mapped to an integer tensor x_q through an affine transformation defined by a *scale* s and a *zero-point* z [12]:

$$x_q = \text{clip} \left(\left\lfloor \frac{x}{s} \right\rfloor + z, q_{\min}, q_{\max} \right), \quad (2.3)$$

where $\lfloor \cdot \rfloor$ denotes rounding to the nearest integer and $[q_{\min}, q_{\max}]$ defines the representable integer range (for example $[-128, 127]$ for signed INT8). During inference, values can be approximately reconstructed by

$$\hat{x} = s(x_q - z). \quad (2.4)$$

The parameters s and z are commonly derived from observed tensor ranges. If x is assumed to lie in $[x_{\min}, x_{\max}]$, a typical choice is

$$s = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}}, \quad z = \left\lfloor q_{\min} - \frac{x_{\min}}{s} \right\rfloor.$$

This formulation is widely used in deployment toolchains because it enables efficient integer arithmetic while approximating the original floating-point computations [12].

Weight quantisation and activation quantisation

Quantisation can be applied to:

- **Weights** (model parameters), which primarily reduces storage and improves cache efficiency. Weight-only quantisation can already provide benefits in memory-constrained scenarios.
- **Activations** (intermediate feature maps), which often dominate runtime memory usage during inference, especially for high-resolution vision models. Quantising activations can substantially reduce memory bandwidth requirements.

In practice, quantising activations is more challenging than quantising weights, because activation distributions depend on the input data and can vary between layers [6].

Granularity: per-tensor vs. per-channel

A key design choice is the granularity at which quantisation parameters are defined [6]:

- **Per-tensor quantisation:** a single (s, z) pair is used for the entire tensor. This approach is simple and efficient, but it can compromise accuracy if the ranges of different channels differ significantly.

- **Per-channel quantisation:** separate scales (and sometimes zero-points) are used for each output channel (common for convolution weights). This typically improves accuracy because it adapts to channel-wise variation at a small additional implementation cost.

For convolutional layers, per-channel quantisation of weights is commonly used in practice because it often yields better accuracy at the same bit-width.

Post-training quantisation (PTQ)

Post-training quantisation converts a trained FP32 model to lower precision without re-training the model. The main advantage of PTQ is that it is fast and easy to apply. A typical PTQ workflow includes a *calibration* step, where a small representative dataset is passed through the network to estimate activation ranges. PTQ is widely used in deployment scenarios when re-training is not feasible [6].

However, PTQ may introduce a noticeable decrease in accuracy for certain models and tasks. Object detection is often more sensitive than image classification because it includes both classification and localisation components and may depend on small numerical differences in bounding box regression.

Quantisation-aware training (QAT)

Quantisation-aware training incorporates quantisation effects during training, allowing the network to adapt to reduced precision. A common approach uses *fake quantisation*: during the forward pass, tensors are quantised and dequantised to simulate INT8 behaviour, while gradients are still computed in floating-point (often using a straight-through estimator). As a result, QAT typically achieves higher accuracy than PTQ at the same bit-width, at the cost of additional training time and complexity [6].

Symmetric vs. asymmetric quantisation

Quantisation can be [6]:

- **Symmetric**, where $z = 0$ and the integer range is centred around zero. This can simplify computation and is common for weights.
- **Asymmetric**, where $z \neq 0$, allowing the range to shift. This can better represent non-zero-centred distributions and is often used for activations.

The choice affects both accuracy and hardware efficiency, and in practice, it is often determined by the deployment framework and accelerator capabilities [6].

Why quantisation helps on edge devices

Quantisation can improve edge inference for several reasons [25]:

- **Reduced model size:** fewer bits per weight lowers storage requirements and improves cache usage.
- **Lower memory bandwidth:** quantised activations require fewer bytes to read and write, which can be critical on constrained devices.
- **Faster compute on supported hardware:** many NPUs and mobile accelerators provide highly optimised INT8 operations.
- **Lower energy consumption:** moving and processing fewer bits often reduces power usage, which is important for battery-powered systems.

At the same time, observed speedups depend on the hardware and runtime stack. If the inference engine does not support a quantised operator, it may fall back to floating-point execution, reducing the practical benefits [25].

Quantisation in the context of compression pipelines

Quantisation is often combined with other compression methods [4]. For example, pruning can reduce the number of channels or filters, and quantisation can then further reduce memory and bandwidth [8]. Knowledge distillation can help recover accuracy lost due to low-precision constraints by training a compact quantised model to mimic a larger teacher [9]. These combinations are commonly used in edge deployment workflows because they address different sources of cost: model size, compute, and numerical precision [4].

2.5.2. Pruning

Pruning refers to a family of methods that reduce the size and computational cost of a neural network by removing parameters that are considered unnecessary [4]. Modern deep learning models are often over-parameterised, meaning that they contain redundancy in their weights and internal representations. Pruning attempts to exploit this redundancy by eliminating parts of the model while maintaining accuracy as much as possible. In the context of edge deployment, pruning is attractive because it can reduce inference latency, memory usage, and energy consumption, especially when it leads to a smaller dense model that runs efficiently on standard hardware [25].

General formulation

Let a model be parameterised by $\theta \in \mathbb{R}^n$. A common view of pruning is to introduce a binary mask $\mathbf{m} \in \{0, 1\}^n$ that selects which parameters remain active:

$$\theta' = \mathbf{m} \odot \theta,$$

where $\odot$ denotes element-wise multiplication and θ' are the effective parameters used at inference. Pruning corresponds to setting some components of $\mathbf{m}$ to zero. The resulting pruned model is then typically fine-tuned to recover performance [8].

Unstructured pruning

Unstructured pruning removes individual weights, for example, by setting to zero weights with small magnitude [8]:

$$m_i = \begin{cases} 0, & \text{if } |\theta_i| < \tau, \\ 1, & \text{otherwise,} \end{cases}$$

where τ is a threshold chosen to reach a desired sparsity level. This approach can substantially reduce the number of non-zero parameters, but the remaining weights form irregular sparse patterns. In practice, unstructured sparsity does not always translate into lower latency on edge devices because efficient execution often requires specialised sparse kernels or hardware support. Without such support, a sparse model may be stored compactly but still executed as if it were dense [25].

Structured pruning

Structured pruning removes whole structures such as convolutional channels, filters, attention heads, or complete blocks. This produces a smaller dense network with reduced tensor dimensions, which typically provides more predictable speedups on common accelerators [25].

For example, in a convolutional layer, one may prune output channels (filters). If the original convolution has C_{out} output channels, pruning removes a subset of these channels and adjusts subsequent layers accordingly. This directly reduces both parameter count and computational cost, since the approximate MAC count of a convolution scales with channel dimensions:

$$\text{MACs} \approx H'W' \cdot C_{\text{out}} \cdot (k^2 C_{\text{in}}).$$

Reducing C_{out} (and often C_{in} in downstream layers) therefore reduces compute and memory traffic [25].

Pruning criteria

Pruning methods differ mainly in how they decide what to remove. Common criteria include [4]:

- **Magnitude-based pruning:** removes weights or channels with small magnitude (simple and widely used).
- **Gradient-based or sensitivity-based pruning:** removes parameters that have a small effect on the loss, often estimated using gradients or second-order approximations.
- **Norm-based channel pruning:** removes channels with small ℓ_1 or ℓ_2 norm in convolution kernels.

In practice, magnitude- or norm-based pruning is popular because it is easy to implement and often provides a good baseline.

One-shot vs. iterative pruning

Pruning can be applied:

- **One-shot**, where a fixed fraction of parameters is removed at once, followed by fine-tuning.
- **Iteratively**, where pruning is performed gradually over multiple stages (e.g., prune a small fraction, fine-tune, repeat). This often yields better accuracy for the same final sparsity because the network has time to adapt [8].

Iterative pruning is especially common when aggressive compression is required, such as for real-time inference on constrained devices.

Pruning and fine-tuning

After pruning, fine-tuning is usually required [8]. Pruning changes the model capacity and can disrupt learned representations. Fine-tuning allows the remaining parameters to adapt and compensate for removed components. In some workflows, pruning is integrated into training by adding sparsity-inducing regularisation, for example, an ℓ_1 penalty on weights or structured regularisers that encourage entire channels to become small.

Pruning for edge deployment

For edge deployment, the practical benefit of pruning depends strongly on whether pruning reduces the *actual* runtime bottlenecks [25]. Structured pruning typically provides more consistent improvements because it reduces tensor sizes and can be accelerated by standard dense

kernels. In contrast, unstructured pruning may yield a smaller parameter count without latency gains unless the target device and inference engine support sparse computation [25].

When applied to object detection models, pruning must be performed carefully because detection performance can be sensitive to capacity reductions, especially for small objects and complex scenes [4]. Common practice is to primarily prune the backbone and neck components (which dominate computation) while preserving critical parts of the detection head. In many cases, pruning is combined with other methods such as quantisation or knowledge distillation to recover accuracy while maintaining a smaller and faster model.

2.5.3. Knowledge distillation

Knowledge distillation is a training strategy used to improve the performance of a compact model by transferring information from a larger, typically more accurate model. The larger model is referred to as the *teacher*, while the smaller model is the *student*. The primary motivation is that small models often have limited capacity and may struggle to achieve high accuracy when trained solely with hard ground-truth labels. A teacher model can provide richer supervision signals that help the student learn a better decision function without increasing inference cost [9].

Teacher–student framework

In the teacher–student setup, the teacher network $f_T(\cdot)$ is trained first (or chosen as an already strong model). The student network $f_S(\cdot)$ is then trained using both:

- **hard labels** (the original targets), and
- **soft targets** derived from the teacher outputs.

Soft targets contain information about class similarities and uncertainties. For example, if the teacher assigns non-zero probability to multiple classes, it communicates that these classes are visually related, which can improve generalisation compared to training only on one-hot labels [9].

Logit distillation and temperature scaling

A common form of distillation uses the teacher and student logits (pre-softmax outputs). Let $\mathbf{z}_T$ and $\mathbf{z}_S$ denote teacher and student logits for the same input. Their softened probability distributions are computed with a temperature parameter $\tau > 1$:

$$p_T(\tau) = \text{softmax}\left(\frac{\mathbf{z}_T}{\tau}\right), \quad p_S(\tau) = \text{softmax}\left(\frac{\mathbf{z}_S}{\tau}\right). \quad (2.5)$$

Higher τ produces a softer distribution, revealing more information about the teacher’s relative preferences among classes. The student is trained by minimising a weighted combination of the standard supervised loss and the distillation loss:

$$\mathcal{L} = (1 - \lambda) \mathcal{L}_{\text{sup}}(y, p_S(1)) + \lambda \tau^2 \text{KL}(p_T(\tau) \parallel p_S(\tau)), \quad (2.6)$$

where $\lambda \in [0, 1]$ balances the two objectives, and $\text{KL}(\cdot \parallel \cdot)$ is the Kullback–Leibler divergence. The τ^2 term is commonly used to maintain comparable gradient magnitudes across different temperature choices [9].

Distillation beyond logits

While logit distillation is widely used for classification, distillation can also transfer information from intermediate representations. Common extensions include:

- **Feature distillation:** the student is encouraged to match the teacher’s feature maps at selected layers.
- **Attention distillation:** the student matches attention maps or activation patterns, which can guide where the model should focus.
- **Relation distillation:** the student learns relationships between samples or regions that are captured by the teacher.

These variants can be particularly helpful when the student architecture differs significantly from the teacher [9].

Knowledge distillation for object detection

Object detection is more complex than classification because the model must solve multiple sub-tasks, typically including classification and localisation of bounding boxes. As a result, distillation for detection often combines multiple signals, such as:

- distillation of classification scores,
- distillation of bounding box regression outputs,
- distillation of intermediate features (e.g., backbone or neck feature maps),
- distillation of region-level predictions (for two-stage detectors) or dense prediction maps (for one-stage detectors).

Because the output structure is richer, detection distillation methods must be designed carefully to avoid transferring teacher errors or overly constraining the student [21].

Why distillation is useful for compression

Distillation is especially useful in compression pipelines because it can reduce the accuracy gap between a compact model and a large baseline without increasing inference cost [4]. This makes it complementary to other efficiency methods:

- A **pruned** model can be fine-tuned with distillation to recover performance lost by removing channels or filters.
- A **quantised** model can be trained with distillation so that reduced precision causes smaller accuracy degradation.
- A **lightweight student architecture** (manually designed or found by NAS) can be distilled from a stronger teacher to improve accuracy under strict latency or energy constraints.

In edge deployment scenarios, where model capacity is limited, distillation is often one of the most effective ways to maintain accuracy while meeting runtime budgets [25].

Limitations and practical considerations

Although distillation can significantly improve compact models, it also introduces additional training complexity [9]. A strong teacher is required, and the choice of distillation targets, temperature τ , and weighting λ can influence results. Moreover, the student may inherit some of the teacher’s biases or failure modes. For these reasons, distillation is typically evaluated in conjunction with other compression techniques under consistent deployment conditions [4].

2.5.4. Neural Architecture Search and Hardware-aware design

Neural Architecture Search (NAS) is a family of methods that automate the design of neural network architectures [3]. Instead of manually choosing the number of layers, channel widths, kernel sizes, or attention configurations, NAS explores a predefined *search space* of candidate architectures and aims to identify models that achieve strong performance. This approach is particularly relevant in the context of efficient deployment, because the architecture itself strongly influences inference latency, memory usage, and energy consumption [25].

Problem formulation

NAS can be expressed as an optimisation problem over architectures α from a search space $\mathcal{A}$. The goal is often to maximise predictive performance while satisfying efficiency constraints:

$$\max_{\alpha \in \mathcal{A}} \text{Acc}(\alpha) \quad \text{subject to} \quad \text{Cost}(\alpha) \leq C_{\max}, \quad (2.7)$$

where $\text{Cost}(\alpha)$ can represent latency, parameter count, memory usage, energy consumption, or a combination of these metrics. In many edge scenarios, the constraint is expressed directly as a latency budget measured on a target device [3].

Alternatively, the objective may be written as a multi-objective trade-off, for example maximising accuracy while penalising cost:

$$\max_{\alpha \in \mathcal{A}} \text{Acc}(\alpha) - \beta \text{Cost}(\alpha),$$

where β controls how strongly efficiency is prioritised.

Search strategies

NAS approaches differ mainly in the strategy used to explore the search space. Common strategies include:

- **Evolutionary search**, where architectures are mutated and selected over generations.
- **Reinforcement learning**, where a controller proposes architectures and receives rewards based on performance.
- **Gradient-based NAS**, where a continuous relaxation of the search space allows optimisation by gradient descent.

In addition to the search strategy, NAS also depends on how candidate architectures are evaluated, which is often the dominant computational cost [3].

Hardware-aware design

In practice, theoretical measures such as FLOPs do not always predict real latency on a specific device. Hardware-aware methods, therefore, incorporate device-dependent measurements or predictors. A hardware-aware formulation replaces $\text{Cost}(\alpha)$ with a metric measured on a target platform (or estimated by a surrogate model):

$$\text{Cost}(\alpha) = \text{Lat}_{\text{device}}(\alpha) \quad \text{or} \quad \widehat{\text{Lat}}_{\text{device}}(\alpha).$$

This is important because practical runtime is influenced by factors such as memory bandwidth, kernel implementations, operator fusion, and the inference engine [25]. For edge deployment, hardware-aware NAS can therefore produce architectures that are efficient *in practice*, not only on paper.

Computational requirements and practical limitations

A major drawback of NAS is its computational cost. Evaluating many candidate architectures can require significant resources, often involving repeated training or fine-tuning. Early NAS approaches were prohibitively expensive, sometimes requiring hundreds or thousands of GPU-days. Although modern methods have improved efficiency (e.g., weight sharing, early stopping, or proxy tasks), NAS can still be difficult to perform under limited computational budgets [3].

For this reason, when compute resources are constrained, NAS is often used in a reduced or practical form, for example:

- restricting the search space to a small set of design choices,
- using pre-trained backbones and searching only parts of the network (e.g., detection head or neck),
- relying on published efficient architectures as baselines rather than performing a full search [26].
- using lightweight proxy metrics or latency predictors instead of measuring every candidate on hardware.

These strategies allow some benefits of architecture optimisation without the full computational cost of large-scale NAS [3].

Relation to model compression

NAS and compression address efficiency in complementary ways. Compression methods such as pruning and quantisation start from an existing architecture and aim to reduce its cost after training [4]. NAS, in contrast, aims to discover architectures that are efficient *by design* [3]. In practice, both approaches can be combined: an architecture designed or selected for efficiency may still benefit from pruning, quantisation, or distillation to further meet strict edge constraints. From an edge deployment perspective, this combination is often attractive because it balances model quality with practical runtime performance [25].

Summary

This chapter introduced the core building blocks used throughout the thesis: neural networks and their training, convolutional networks for vision workloads, and transformer-based models,

including vision transformers, as modern alternatives for sequence-style processing in vision [13, 27, 5].

It then motivated *frugal artificial intelligence* as a practical viewpoint where accuracy must be balanced against latency, memory, energy, and data/communication constraints typical of edge deployment [25]. To meet these constraints, the chapter reviewed four main families of efficiency methods: quantisation (lower numerical precision), pruning (removing parameters/structures), knowledge distillation (teacher–student transfer), and hardware-aware NAS (architectures optimised under device-specific cost metrics). Finally, it highlighted that real deployment performance depends on the full hardware–software stack (e.g., operator support, memory bandwidth, runtime kernels), so evaluation should reflect realistic conditions rather than relying only on proxy metrics.

Bibliography

- [1] Areeba Abid, Priyanshu Sinha, Aishwarya Harpale, Judy Gichoya, and Saptarshi Purkayastha. Optimizing medical image classification models for edge devices. In Kenji Matsui, Sigeru Omatu, Tan Yigitcanlar, and Sara Rodríguez González, editors, *Distributed Computing and Artificial Intelligence, Volume 1: 18th International Conference*, volume 327 of *Lecture Notes in Networks and Systems*, pages 77–87. Springer, Cham, 2022.
- [2] Aphex34. Typical cnn.png, December 2015. Image. Source: Own work. Licensed under CC BY-SA 4.0 (<https://creativecommons.org/licenses/by-sa/4.0/>).
- [3] Han Cai, Ligeng Zhu, and Song Han. Proxylessnas: Direct neural architecture search on target task and hardware. In *International Conference on Learning Representations (ICLR)*, 2019.
- [4] Yu Cheng, Duo Wang, Pan Zhou, and Tao Zhang. A survey of model compression and acceleration for deep neural networks. *arXiv preprint arXiv:1710.09282*, 2017.
- [5] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.
- [6] Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. A survey of quantization methods for efficient neural network inference. *arXiv preprint arXiv:2103.13630*, 2021.
- [7] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems*, 2014.
- [8] Song Han, Huizi Mao, and William J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. In *International Conference on Learning Representations (ICLR)*, 2016.

- [9] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- [10] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [11] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017.
- [12] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew T. H. Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [13] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.
- [14] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [15] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [16] Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015.
- [17] Warren S. McCulloch and Walter Pitts. A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, 5(4):115–133, 1943.
- [18] Andrew Miller, Fabio Di Troia, and Mark Stamp. An empirical analysis of hidden markov models with momentum. In Mark Stamp and Martin Jureček, editors, *Machine Learning, Deep Learning and AI for Cybersecurity*, pages 169–206. Springer, Cham, 2025.
- [19] Boris T. Polyak. Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964.
- [20] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

- [21] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems*, 2015.
- [22] Herbert Robbins and Sutton Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400–407, 1951.
- [23] Frank Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958.
- [24] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.
- [25] Vivienne Sze, Yu-Hsin Chen, Tien-Ju Yang, and Joel Emer. Efficient processing of deep neural networks: A tutorial and survey. *arXiv preprint arXiv:1703.09039*, 2017.
- [26] Mingxing Tan, Ruoming Pang, and Quoc V. Le. Efficientdet: Scalable and efficient object detection. *arXiv preprint arXiv:1911.09070*, 2020.
- [27] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

List of Figures

1.1	Variants of edge devices (reproduced from [1]).	12
2.1	Stochastic Gradient Descent comparison (reproduced from [18]).	16
2.2	CNN architecture visualisation. Reproduced from Aphex34 [2] (CC BY-SA 4.0). .	18
2.3	Multi-head attention overview (reproduced from [27]).	21
2.4	Transformer architecture overview (reproduced from [27]).	22